VulnBump: A Benchmark for Fixing Vulnerable Dependencies in Real Projects While Keeping the Build and Tests Green

Gabija Pasiunaite[1], Ryan McMorrow[1], Brian Clark[1], Liran Tal[1], and Luke Tu[2,1]

[1]Snyk
[2]Brown University

October 12, 2026

Abstract

Software composition analysis (SCA) tools report far more vulnerable dependencies than teams fix: in Snyk Open Source production data, 94% of vulnerability findings have a known fixed version, yet only about 10% are resolved. Agentic coding tools could close this gap, but no benchmark evaluates them on vulnerability-driven upgrades in real, runnable projects across ecosystems, verified independently of the agent. We present VulnBump: 307 upgrade tasks covering 2,066 fixable vulnerabilities in 43 repositories across six package managers, matched to Snyk Open Source production vulnerability distributions. An upgrade passes only if, in a fresh container, dependencies resolve, a rescan shows the targeted vulnerabilities fixed, the project builds, and every previously passing test still passes. A non-agentic baseline that only applies Snyk's recommended version passes 46.9% of upgrades; with the same model (Claude Sonnet 5.5), Snyk's Evo Remediation Agent passes 90.2%, Claude Code + Snyk scan 77.5% and Claude Code unaided 59.6%. The agents fail in opposite ways: Claude Code mostly leaves the vulnerability in place, whereas most of Evo Remediation Agent's failures break the build or tests. An SCA scanner is a key tool for agents: Snyk's guidance raises Claude Code's PASS rate by 17.9 percentage points; without it, Claude Code often settles on a still-vulnerable version. Yet knowing the target is not enough: given the same Snyk findings, Evo Remediation Agent's purpose-built harness passes 12.7 points more than Claude Code + Snyk scan. Major framework migrations remain hard for all harnesses. Because the agents keep projects working at similar rates, evaluating remediation quality in dependency upgrades requires verifying the fix, the build and the tests together.

1 Introduction

Software composition analysis (SCA) scanners such as Snyk Open Source [21] flag vulnerabilities in open-source dependencies, and the fix is usually an upgrade: in Snyk production data, 94.0% of findings have a known fixed version, yet only about 10% are resolved (Section 3.1). The bottleneck is upgrading *safely*. Of 306 dependency-update pull requests that bots such as Dependabot opened in Maven projects and developers rejected after discussion, 51.3% involved incompatibility concerns (compile, test or CI failures, dependency conflicts) [24]. In Snyk's data, about a quarter of fixes need a major-version jump and most vulnerable packages arrive transitively (Section 3.1), and even minor and patch releases often contain breaking changes [12, 14]. Cautious upgrades do not solve the problem: non-major upgrades of direct dependencies fix only 25.71% of CVEs [24]. An upgrade that removes the vulnerability but breaks the build or tests is not a fix.

Agentic coding tools could do this work end to end, but no benchmark measures whether they do it reliably. That requires (i) real repositories with working builds and tests, (ii) tasks driven by real vulnerability findings with known fix paths, and (iii) verification, independent of the agent, that the vulnerability is gone *and* the project still works. Existing benchmarks cover at most two (Section 2): breaking-update benchmarks [10, 17, 23] are not driven by vulnerabilities, and CORAL [24] scores against its own CVE-to-version mapping rather than a production scan.

We introduce VulnBump to fill this gap. Each task, an *upgrade*, pairs a real repository, pinned at a vulnerable commit, with one upgrade recommended by a Snyk scan. The simplest fix is a version bump, which raises the dependency's version in the manifest or lockfile and changes nothing else; harder ones also need code changes or dependency conflict resolution. A verifier re-checks the agent's work in a fresh container with four *gates*: dependencies resolve, a rescan shows the targeted vulnerabilities fixed, the project builds, and every previously passing test still passes. Only an attempt that passes all four is a PASS.

We use VulnBump to evaluate four harnesses on the same 307 upgrades. The package manager bump, the non-agentic baseline, bumps the package to Snyk's recommended version with the project's own package manager, and changes nothing else. The three agentic harnesses use the same model, Claude Sonnet 5.5: Snyk's Evo Remediation Agent, and the general-purpose Claude Code [1] run with the Snyk scan and recommended upgrade (Claude Code + Snyk scan) and without them (Claude Code unaided). The baseline passes 144 upgrades, a lower bound on what mechanical upgrading can solve (Section 5.3). On these *Snyk-advice-solvable* upgrades, an agent can only keep or lose an easy fix; on the other 163, it must do more than bump a version, such as adapting code to breaking changes or resolving dependency conflicts.

Two design choices let VulnBump tell these harnesses apart. First, because each task starts from a vulnerability, VulnBump evaluates remediation directly and can therefore measure *avoidance*, an agent leaving the vulnerability in place rather than taking a risky upgrade. Breaking-update benchmarks cannot detect avoidance, yet it is Claude Code's most common failure (Section 5.3). Second, a PASS requires both a remediated vulnerability and a working project. Evaluated on dependency resolution, build and tests alone, the agents perform similarly; only the remediation check distinguishes them.

This paper makes three contributions:

- **VulnBump, a dataset grounded in production data:** 43 repositories, selected to match Snyk production vulnerability distributions from 99 qualifying candidates with a reproducible install, build and test baseline, yielding 307 upgrade tasks over 2,066 fixable vulnerability instances, from bumps that need no code changes to major migrations.
- **A verifier that does not trust the agent**, applying the four gates in a separate container, while the agent runs in an isolated, production-like sandbox with leakage controls.
- **A same-model comparison** of a purpose-built remediation engine and a general-purpose coding agent, with per-gate results, paired statistics, a failure analysis that separates avoidance from breakage, and the package manager bump baseline.

Our main findings are:

- **Only the vulnerability check separates the agents.** On dependency resolution, build and tests alone, the three agents perform similarly (92.8–93.8%); with the rescan, Evo Remediation Agent passes 90.2% of upgrades, Claude Code + Snyk scan 77.5% and Claude Code unaided 59.6%, a ranking that holds per repository (Section 5.1).

- **An SCA scanner is a key tool for agents fixing vulnerabilities.** Snyk's recommended upgrade applied by the package manager bump passes 46.9% of upgrades and Claude Code without Snyk 59.6%; together, as Claude Code + Snyk scan, they pass 77.5%. Without an SCA scan, Claude Code regresses on 20 of the 144 upgrades the package manager bump solves, 15 of them by choosing a version Snyk still flags: web search alone does not reliably find a safe version (Section 5.2).
- **Knowing the target is not enough.** Given the same Snyk findings and model, Evo Remediation Agent's purpose-built harness passes 12.7 percentage points more than Claude Code + Snyk scan, mostly on the 163 upgrades the package manager bump fails, where Evo Remediation Agent passes 82.2%, Claude Code + Snyk scan 61.3% and Claude Code unaided 36.2% (Sections 5.1 and 5.2).
- **The agents fail in opposite ways.** Claude Code mostly stops short and fails silently: the vulnerability remains in 107 of Claude Code unaided's 124 failures, 70 of which it claimed it fixed, and in 48 of Claude Code + Snyk scan's 69. Evo Remediation Agent usually lands the upgrade, so 21 of its 30 failures break the build or tests, where the project's own checks catch them (Section 5.3).
- **Major framework migrations remain hard.** PASS rates fall from 96%, 88% and 70% on non-major upgrades to 81%, 61% and 42% on major ones for Evo Remediation Agent, Claude Code + Snyk scan and Claude Code unaided; half of Evo Remediation Agent's major PASSes edit application source, and 23 upgrades, mostly Spring, Spring Boot and Dropwizard migrations, defeat all three agents (Section 5.4).

2 Related Work

Existing upgrade benchmarks check that a project still works after an upgrade but not that a vulnerability was remediated, and the one vulnerability-driven evaluation covers a single ecosystem. Table 1 compares the closest benchmarks on the checks they use to score an attempt.

Breaking dependency updates. Benchmarks of breaking dependency updates judge an upgrade by whether the project still builds and passes its tests. BUMP [17] collects 571 reproducible breaking updates from 153 Maven projects, each a one-line version bump of a declared dependency that fails the build. BUMP defines no repair scoring of its own; its marks in Table 1 reflect the `mvn clean test` run that reproduces each breaking update. Byam [18] repairs BUMP's compilation errors with LLM prompts, and BigBag [16] has an agent generate reusable AST transformation rules for them, on a subset of 157 BUMP upgrades. Tests generated by BreakGuard [15] to pass before and fail after an update detect at best 30.3% of BUMP breaking updates, almost all crashes rather than behavioural changes.

Recent benchmarks apply the same check to LLM repair across ecosystems. DepRepair [23][1] curates 95 instances of 10 well-known major migrations across Maven, npm, Cargo and PyPI; all its methods, including the agents, generate patches from about ten pre-scoped files without an in-loop build or test environment. DepBench [10] mines 203 tasks from bot pull requests that needed human source or test changes, across five ecosystems, and hides its held-out upgrade tests from the agent; its best configuration solves 51.2% of tasks, and changing only the harness moves results by up to

[1]Yang et al. name their benchmark "DepBench" and their method DepRepair; we use "DepRepair" for the benchmark to avoid confusion with the benchmark of Luo et al. [10].

Table 1: Dependency-upgrade and remediation benchmarks compared with VulnBump on the checks used to score an attempt. ✓ = scored; ∘ = partial, e.g. checked only within another step or against the authors' own CVE mapping (CORAL); Vuln = rescan with a production SCA scanner; † = held-out tests hidden from the agent. BUMP defines no repair scoring; LADU builds and tests in its loop but scores only overlap with manual upgrades.

Benchmark	Ecosystems	Size	Task trigger	Real projects	Dependencies	Build	Tests	Vuln
BUMP [17]	Maven	153 repositories, 571 upgrades	Breaking update	✓	∘	✓	✓	
BigBag [16]	Maven	157 upgrades	Breaking update	✓	∘	✓	✓	
DepRepair [23]	Maven, npm, Cargo, PyPI	95 upgrades	Major migration	✓		∘	∘	
DepBench [10]	npm/Yarn, Maven, Go, Cargo, Python	203 upgrades	Bot upgrade PR	✓	∘	∘	✓†	
CORAL [24]	Maven	301 repositories, 750 unique CVEs	CVE	✓	∘	✓	✓	∘
REQBench [8]	pip	34 repositories, 2,095 upgrades	Library upgrade	✓	∘	∘	∘	
LADU [22]	Maven	3 synthetic repositories	Major upgrade					
VulnBump	Maven, Gradle, npm, Yarn, pip, Poetry	43 repositories, 307 upgrades, 2,066 vulnerabilities	Vulnerability fix path	✓	✓	✓	✓	✓

16.3 points. PCREQ [8] infers compatible requirements for Python library upgrades to every later version. On its REQBench dataset, LLM baselines solve only 11–24% of the 406 scenarios in which a plain `pip` upgrade crashes at runtime, yet fail 9–13% of those `pip` already handles: the trade-off between fixing hard cases and breaking easy ones that VulnBump measures against its package manager bump (Section 5.2). None of these benchmarks starts from a vulnerability or checks that one was removed, so none can observe an agent that leaves it in place.

Vulnerability remediation. To our knowledge, the only prior evaluation that starts from vulnerable dependencies is CORAL [24], and it covers only the Java (Maven) ecosystem. It uses the Z3 SMT solver to select compatible versions for vulnerable dependencies in 301 Maven projects and reports fixing 87.56% of CVE occurrences. It evaluates a solver rather than agents, never edits application code, and scores all tools against the authors' own NVD-derived CVE-to-version mapping, which CORAL also optimises against, rather than a production SCA scan.

What VulnBump adds. VulnBump tasks start from vulnerability remediation: several releases often fix a vulnerability, but the agent must remediate the vulnerability while keeping the build

and tests intact. Taking the smallest fixing release does not guarantee this: security fixes are often patch or minor releases, which semantic versioning promises are backward compatible, but this promise often fails. On Maven Central, 35.7% of minor and 23.8% of patch releases contain at least one breaking change [14]; a later replication found that 83.4% of upgrades conform to semantic versioning, yet 20.1% of non-major upgrades still introduce breaking changes, although only 7.9% of sampled clients would actually break [12]. In Go, Li et al. [9] found that 86.3% of upgrades comply with semantic versioning, yet 28.6% of non-major upgrades introduce breaking changes. Even a minimal, in-range upgrade can therefore break a consumer, and adapting code to a given library version is itself hard for models: the largest enterprise models reach only 48–51% baseline success on GitChameleon 2.0's [11] 328 version-conditioned Python problems, and LibEvoBench [4], which tests whether models use the correct API for a given version of PyTorch, NumPy or SciPy, finds them largely version-oblivious: stating the target version in the prompt does not improve accuracy.

No existing benchmark measures this joint task on real-world projects across multiple ecosystems. VulnBump checks dependency resolution, the targeted vulnerabilities, the build and the tests together in six package managers (Table 1), and includes transitive-only fixes, which BUMP's one-line bumps of declared dependencies exclude. As SWE-bench [7] did for GitHub issues, it evaluates on real repositories, with agents that build and test in the loop. LADU [22] also compiles and tests in its loop while adapting a Java project to a chosen version of an industrial framework, but is evaluated on three synthetic repositories by overlap with manual upgrades. Finally, VulnBump reports per-gate results, a first-failed-gate breakdown and repository-clustered significance tests, which none of BUMP, CORAL, REQBench, DepRepair or DepBench report.

3 The VulnBump Dataset

VulnBump is shaped by Snyk Open Source production data, which determined the ecosystems we mined and the vulnerability profile we aimed for. From the mined open-source candidates, we kept repositories that build reproducibly, pass more than 85% of their tests and contain at least one vulnerability, and selected those that most closely match the production distributions.

3.1 Grounding the Dataset in Snyk Production Data

The target distributions come from Snyk Open Source findings from April to September 2026: about 300,000 organisations and 459 million unique *issue–project pairs*. A Snyk project is one monitored manifest, and a repository can contain several. Because each project must be fixed separately, a vulnerability counts once for every project it affects. In this data, 94.0% of findings have a known fixed version, and for 93.7% Snyk also finds an upgrade path to it within the project, yet 89% are still open, about 10% resolved and 1% ignored. Four Snyk Open Source production observations shaped VulnBump:

- **Package-manager coverage.** Java (62.7%), JavaScript (22.1%) and Python (9.9%) account for almost 95% of findings; VulnBump targets the six package managers that cover about 92% of pairs (Maven, Gradle, npm, Yarn, pip and Poetry).
- **Direct versus transitive.** In a 1% sample of projects (4.2 million findings), most findings are transitive: the vulnerable package is declared in the manifest for only 14.0% of Maven, 19.4% of Gradle, 34.6% of npm, 22.1% of Yarn and 15.3% of Poetry findings. The exception is pip (55.5%), because `requirements.txt` files often list the full resolved set.

- **Severity.** High (47.5%) and medium (40.8%) findings dominate; critical is 7.1% and low 4.6%.
- **Fix difficulty.** The smallest semantic-versioning jump to a fixed release splits patch/minor/major 40/33/26% in Java, 27/48/25% in JavaScript and 21/50/29% in Python: about a quarter of fixes need a major upgrade in every language.

3.2 Candidate Repository Collection

Candidates come from five sources covering all six package managers (counts after deduplication):

- **BUMP projects (Maven, 133 candidates)** [17], pinned at the parent of a breaking-update commit, where the project still builds and passes its tests.
- **Popular Maven projects (34):** top-starred Maven projects on GitHub, pinned at their last commit before 21 May 2022. This mirrors CORAL's setup of popular Maven projects at its snapshot date [24], and leaves them old enough to contain open vulnerabilities.
- **Popular open-source projects (274):** curated applications, frameworks, starters and tutorials (Gradle, npm, Yarn, pip, Poetry). Well-maintained projects often have no open vulnerabilities at their latest commit, so many are pinned at an older one.
- **GitHub searches for vulnerable packages (280):** latest commits of manifests that pin known-vulnerable versions of frequently vulnerable packages (e.g. `axios`, `urllib3`), plus repository searches for less-represented package managers.
- **Gradle submodules (46):** modules of large multi-project builds (at the parent's commit), scanned separately so that one task does not span hundreds of modules.

We scan each candidate with `snyk test --json` and record, per unique issue–project pair, severity, fixability, direct or transitive status and fix difficulty, the semantic-versioning jump from the installed version to the earliest fixed release. Some repositories were collected at several commits, giving 805 candidates from 721 repositories. We kept the most vulnerable commit of each, which leaves 767 candidates: one per repository plus the 46 Gradle submodules.

3.3 Filtering for Reproducible and Vulnerable Baselines

So that any later breakage can be attributed to the upgrade, we keep only candidates with a reproducible baseline: the unmodified project must install, build and pass its tests from a clean checkout in its ecosystem's image (JDK 17, Node 20 or Python 3.11; Section 4.3). Each step had a 30-minute limit, and each candidate had to pass five filters (Figure 1):

1. **Dependencies resolve:** every declared dependency can be downloaded and installed with the project's own package manager (e.g. `mvn dependency:resolve`, `pip install`), without private registries (508 of 767 pass).
2. **Build:** Maven and Gradle projects compile; other ecosystems have no build step (475).
3. **Tests run:** at least one test is discovered and executed (199).
4. **>85% tests green** on the unmodified project (141).
5. **Vulnerable:** Snyk Open Source reports at least one vulnerability (99).

The largest drop is at the third filter: many projects have no tests that run in a clean environment, and most pip projects need external services such as Postgres or Redis. Earlier failures were mostly ecosystem-specific: multi-module resolution, private registries and plugins (Maven, Gradle), peer-dependency conflicts (npm), and lockfile or Python-version mismatches (Poetry). This leaves 99 qualifying candidates from 97 repositories (55 Maven, 9 Gradle, 19 npm, 8 Yarn, 3 pip and 5 Poetry).

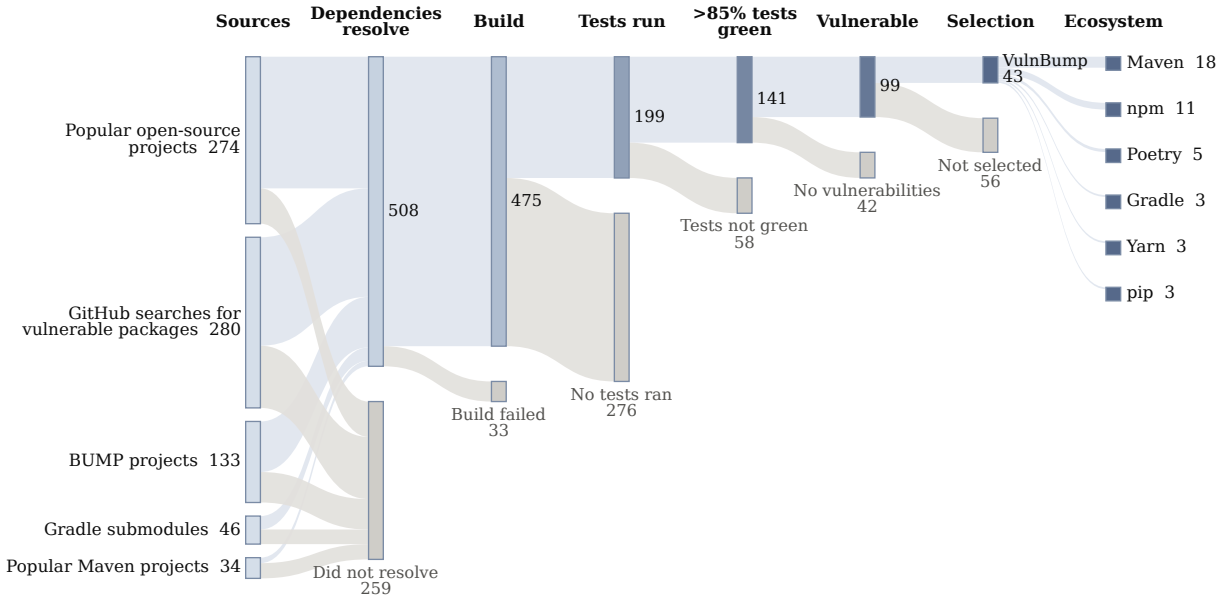


Figure 1: Dataset selection: the 767 deduplicated candidates through the five filters (passing on top, dropped in grey), and the 43 selected repositories by package manager.

3.4 Repository Selection

From the 99 qualifying candidates we selected 43 repositories that match Snyk production data in direct versus transitive share, vulnerabilities per repository, severity and fix difficulty, while giving each package manager enough cases to report on rather than weighting strictly by production share. The final VulnBump dataset comprises 18 Maven, 3 Gradle, 11 npm, 3 Yarn, 3 pip and 5 Poetry repositories (21 Java, 14 JavaScript, 8 Python), with a median of 32 unique vulnerabilities (range 1–252) and 88 passing tests; 19 repositories have at least 1,000 GitHub stars.

The dataset matches production within a few points on severity, fix difficulty, and on package-manager share except for Gradle, and on direct share for Maven, npm, Yarn and Poetry (Figure 2). Vulnerabilities per project span a similar range for Java and JavaScript, while the Python repositories cluster around a higher median.

Two deviations from Snyk Open Source production distributions remain. First, Gradle is under-represented (3% of vulnerabilities versus 17%), with a lower direct share (11% versus 19%): most Gradle candidates failed the filters, nearly half of them at dependency resolution, and the largest qualifying ones were too large to evaluate (Section 7). Second, pip vulnerabilities are mostly direct (84% versus 56%), because only three pip repositories passed the baseline filtering.

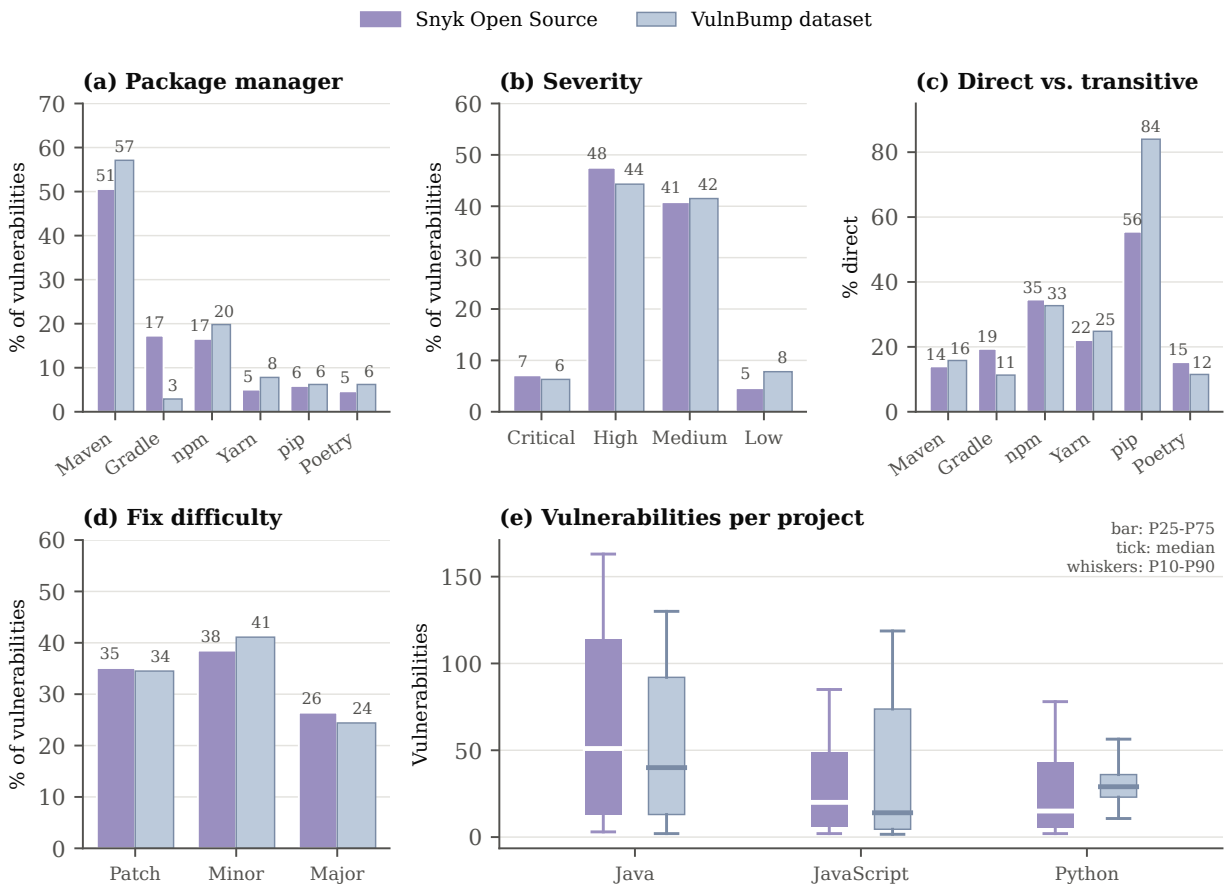


Figure 2: Snyk Open Source production data (April–September 2026) versus VulnBump dataset distributions (43 repositories). Production fix difficulty covers Java, JavaScript and Python findings. Production direct shares come from a 1% random sample of projects.

4 Remediation Task and Fix Verification

This section defines the upgrade task, the four harnesses, the controlled sandbox in which they run, the verification gates and the metrics; Figure 3 shows how they fit together.

4.1 Remediation Task

Each task asks an agent to apply one package upgrade (or pin) recommended by a baseline Snyk scan (`snyk test --all-projects --ignore-policy`) of the installed, pinned repository, which must first install, scan, build and pass at least one test at evaluation time. Snyk groups fixable vulnerabilities into *fix groups*: one upgrade or pin in one manifest, with the vulnerabilities it fixes. Vulnerabilities without a fix path are never counted against an agent.

We call each fix group an *upgrade*; it is the unit of a VulnBump task, so a repository whose 50 fixable vulnerabilities require upgrading 20 packages yields 20 upgrades. Within an upgrade, we count *fixable vulnerability instances*: issue–project pairs in the baseline scan for which Snyk offers an upgrade or pin. If a vulnerability is reachable through several paths, each fixed by a different upgrade, it counts as fixed only when every one of those upgrades removes its path (Listing 1).

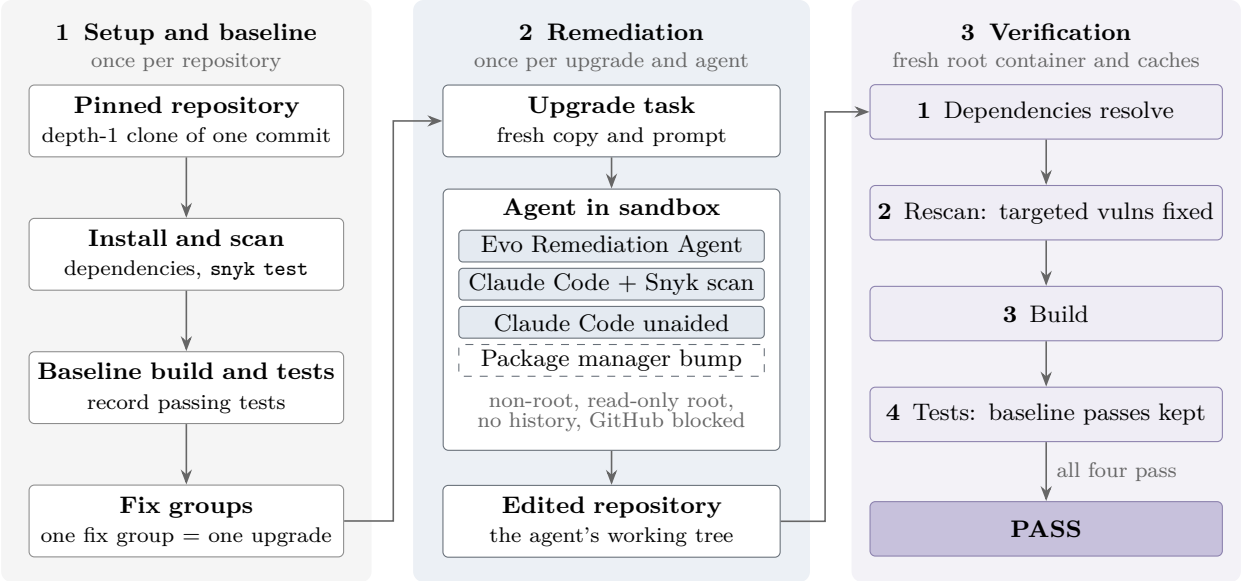


Figure 3: The VulnBump pipeline: (1) Setup and baseline: the pinned repository is installed, scanned, built and tested, and every Snyk fix group becomes one upgrade; (2) Remediation: once per upgrade, the agent edits a fresh repository copy in a sandbox, while the package manager bump (dashed) only applies Snyk's recommended version and refreshes the lockfile; (3) Verification: a fresh container applies four gates to the agent's working tree; PASS requires all four.

Listing 1: Baseline `snyk test` output for one VulnBump repository (expressjs/generator). Its seven vulnerabilities form four upgrades, one per "Upgrade" block; the `mkdirp` upgrade fixes a transitive vulnerability (`mkdirp > minimist`), and `SNYK-JS-MINIMIST-2429795` is reachable through both the `minimist` and `mkdirp` upgrades, so it counts as fixed only if both packages are bumped.

```
Tested 9 dependencies for known issues, found 7 issues, 8 vulnerable paths.

Issues to fix by upgrading:

  Upgrade ejs@2.6.2 to ejs@3.1.10 to fix
  ×Improper Control of Dynamically-Managed Code Resources [Medium Severity][SNYK-JS-EJS-6689533] in ejs@2.6.2
    introduced by ejs@2.6.2
  ×Remote Code Execution (RCE) [High Severity][SNYK-JS-EJS-2803307] in ejs@2.6.2
    introduced by ejs@2.6.2

  Upgrade minimatch@3.0.4 to minimatch@3.1.3 to fix
  ×Regular Expression Denial of Service (ReDoS) [Medium Severity][SNYK-JS-MINIMATCH-3050818] in minimatch@3.0.4
    introduced by minimatch@3.0.4
  ×Regular Expression Denial of Service (ReDoS) [High Severity][SNYK-JS-MINIMATCH-15309438] in minimatch@3.0.4
    introduced by minimatch@3.0.4
  ×Inefficient Algorithmic Complexity [High Severity][SNYK-JS-MINIMATCH-15353389] in minimatch@3.0.4
    introduced by minimatch@3.0.4

  Upgrade minimist@1.2.5 to minimist@1.2.6 to fix
  ×Prototype Pollution [Low Severity][SNYK-JS-MINIMIST-2429795] in minimist@1.2.5
    introduced by minimist@1.2.5 and 1 other path(s)

  Upgrade mkdirp@0.5.1 to mkdirp@0.5.5 to fix
  ×Prototype Pollution [Low Severity][SNYK-JS-MINIMIST-2429795] in minimist@1.2.5
    introduced by minimist@1.2.5 and 1 other path(s)
  ×Prototype Pollution [Medium Severity][SNYK-JS-MINIMIST-559764] in minimist@0.0.8
    introduced by mkdirp@0.5.1 > minimist@0.0.8

Package manager: npm
Target file: package-lock.json
```

Each upgrade is attempted from a fresh copy of the pinned repository and scored on its own (*single-upgrade* mode), so one upgrade's failure cannot affect another.

4.2 Harnesses

Package manager bump. The package manager bump, our non-agentic baseline, applies Snyk's recommended upgrade and nothing else: it uses no model, edits no code or tests, and never tries another version. It sets the named package's version with the ecosystem's own mechanism (`npm install pkg@ver`, `yarn add`, `poetry add`, or the version in `requirements.txt`, `pom.xml` or the Gradle build), and pins an undeclared package the same way, e.g. with an explicit Maven `<version>` or a Gradle `constraints` entry. A pin is *managed* when it overrides a version set by a Maven parent or BOM or by Spring's dependency-management plugin. If the lockfile refresh fails, so does the dependency gate; the only fallback is retrying npm with `--legacy-peer-deps`. Python packages are pinned exactly (`==`); pinning with `>=` changes the outcome of 2 of the 60 Python upgrades and leaves the total unchanged.

Agentic harnesses. All three agents use the same model (`claude-sonnet-5-5` on Vertex AI) and a limit of 110 agent turns per upgrade. They differ in harness, prompt, tools and Snyk information, so we compare complete systems as a user would get them, rather than ablating one component.

Evo Remediation Agent is Snyk's agentic remediation engine, available through the Snyk CLI and a cloud service. It leverages *Snyk intelligence*, which evolved from Snyk's deterministic SCA fix engine for pull requests [19]. We invoke the CLI [20] once per upgrade with the IDs of the vulnerabilities that upgrade fixes:

```
snyk fix --agentic --sca --auto-approve --prepare-env --issue-ids <ids>
```

With `--auto-approve`, it applies every fix without prompting, including upgrades Snyk rates as likely to break the build, which a user could otherwise skip. It works from the same Snyk findings as Claude Code + Snyk scan, runs its own scan, the build and tests, and repairs errors before finishing.

Claude Code + Snyk scan is Claude Code 2.1.289, prompted with the project's test command and the upgrade's "`Upgrade X to Y to fix`" block from the baseline report, which names the target version. It may run Snyk Open Source [21] (`snyk test`).

Claude Code unaided gets the same prompt without the target version ("`Upgrade X to fix these vulnerabilities:`") and no Snyk access, so it must find a fixed version itself, e.g. from OSV [13].

4.3 Sandbox and Leakage Controls

The agent runs in a restricted container modelled on Evo Remediation Agent's production sandbox: a non-root user, a read-only filesystem except for the repository and a scratch directory, and 24 GB of memory. The ecosystem image (JDK 17 with Maven or Gradle, Node 20, or Python 3.11 with pip or Poetry) adds Node.js, a shell, `jq` and `yq`, and the project's dependencies are installed beforehand. Package registries are reachable (Maven Central through a read-through cache); Gradle wrapper distributions are pre-seeded because GitHub is blocked. Claude Code runs headless, without permission prompts, with its standard tools [2] (shell; file read, edit and write; web fetch and web

search). Evo Remediation Agent has its own tools to read, edit and run commands, and no web tool.

Three controls limit leakage of the eventual fix:

- **No commit history.** Each repository is a depth-1 clone of the pinned commit, with no remote, refs or tags.
- **No GitHub.** GitHub and mirrors that serve its repository files (jsDelivr, raw.githack, Sourcegraph) are blocked, and Claude Code has matching `WebFetch` deny rules and no `gh` CLI. Two repositories whose setup fetches from GitHub are exempt from the host block; only the `WebFetch` rules apply there.
- **No Snyk for Claude Code unaided.** It gets no Snyk CLI or token: `snyk.io`, `security.snyk.io`, `api.snyk.io` and `app.snyk.io` are blocked, and links to `security.snyk.io` are removed from its prompt.

The rest of the web, including changelogs and migration guides hosted outside GitHub, stays open; release notes hosted on GitHub are blocked with the rest of GitHub. Section 7 discusses residual risks.

4.4 Verification

Every fix is verified independently, without relying on the agent's report. The verifier checks the agent's working tree as root in a fresh container, with fresh caches and output directory, so nothing the agent left outside the repository (e.g. PATH shims or a local Maven repository) can affect the evaluation metrics. Verification runs all four gates in order, even when an earlier gate fails:

1. **Dependency resolution**: the edited manifests and lockfiles install with the strict command CI would run, so session-only workarounds (e.g. an unsaved install flag) do not count.
   - *Commands:* `mvn dependency:resolve`, `npm ci`, `yarn install --frozen-lockfile`, `poetry install`, or `pip install -r requirements.txt` plus the project; `npm install` for the three npm repositories without a committed lockfile.
   - *Java:* Gradle resolves dependencies during the build, so its resolution failures surface at the build gate; for Maven, which settles version conflicts itself, the gate is effectively a build precondition.
   - *Fallback:* if the strict install fails, npm and Yarn repositories install leniently (`npm install --legacy-peer-deps`, `yarn install`) so that the remaining gates can still run.
2. **Rescan**: in a post-fix `snyk test` of every target manifest, the targeted vulnerabilities count as fixed when they are no longer reachable through the upgraded package or through any newly introduced dependency path. A rescan that fails or omits a target manifest fails the gate. Both scans use `--ignore-policy`, so a `.snyk` file cannot hide a vulnerability.
3. **Build**: Maven and Gradle projects compile (`mvn compile`, `gradlew compileJava`); JavaScript projects run their TypeScript type-check (`tsc --noEmit`) or build script; Python projects import their main package, catching removed modules or symbols. Seven JavaScript repositories have neither a type-check nor a build script, so only their tests catch breakage.

4. **Tests**: every baseline-passing test still passes; a skipped or missing test is a regression. Tests are matched by name from JUnit XML (Maven, Gradle, pytest) or Jest JSON reports; for the six Mocha-based npm repositories without JUnit output, the passing count in the log must not drop below the baseline's, and a log with no count fails. Agents may adapt tests to breaking changes (renames, API migrations, imports) but must not edit tests to make them pass.

An attempt is a **PASS** only if all four gates pass. For the `minimist` upgrade in Listing 1, `npm ci` must accept the edited `package-lock.json`, the rescan must no longer find `SNYK-JS-MINIMIST-2429795` on a path through `minimist`, the build gate does not apply, and Mocha must still report at least the baseline's 139 passing tests.

4.5 Metrics

Gate pass rates. The share of the 307 upgrades that pass each gate individually, and all four gates (the PASS rate). A gate that does not apply, such as dependency resolution for Gradle, counts as passed.

Vulnerability-fix rate. The share of the 2,066 fixable vulnerability instances fixed (Section 4.1); an instance whose rescan failed counts as not fixed.

Per-repository (macro) rates. Repositories contribute 1 to 46 upgrades, so pooled rates, computed over all 307 upgrades together, lean towards a few large repositories. Macro rates average per-repository rates; we report them for the PASS rate.

Paired comparisons. For two agents on the same upgrades, we test the macro PASS-rate difference for significance with a cluster bootstrap over repositories [5, 6] (10,000 resamples), which accounts for correlation within a repository.

Snyk-advice-solvable split. The upgrades the package manager bump passes, a lower bound on what mechanical upgrading can solve (Section 5.3). An agent's *regressions* are upgrades the package manager bump passes and the agent fails; its *gains*, upgrades it passes and the package manager bump fails. We report PASS rates on both sides.

5 Results

All four harnesses ran on the same 307 upgrades from 43 repositories. The three agents share identical baselines: 2,066 fixable vulnerability instances in their baseline Snyk scans. The package manager bump ran 3 days later on the same pinned repositories, and its 307 upgrades pair one-to-one with the agents': 305 have the same recommended upgrade, and two the same package with a newer target version (Section 8). There were no infrastructure errors or wall-clock timeouts; four failed verification rescans (one for the package manager bump, one for Claude Code + Snyk scan, two for Claude Code unaided) count as failures.

5.1 Headline Results

Evo Remediation Agent passes every gate on 90.2% of upgrades, 12.7 percentage points more than Claude Code + Snyk scan and 30.6 more than Claude Code unaided, and fixes 95.9% of vulnerability instances (Table 2). The Snyk scan and recommended upgrade raise Claude Code's PASS rate by 17.9 points. The package manager bump, which applies Snyk's recommended upgrade and nothing else, passes 46.9% and trails every agent on every gate.

Table 2: Key results on VulnBump, with Claude Sonnet 5.5 for all agents; the package manager bump is the non-agentic baseline. Gate and PASS rates are out of 307 upgrades. *Targeted vulnerabilities fixed* is the rescan gate. Vulnerability instances are out of 2,066 (Section 4.5).

	Package manager bump	Claude Code unaided	Claude Code + Snyk scan	Evo Remediation Agent
Dependencies resolve ↑	82.4%	95.4%	94.8%	**100%**
Targeted vulnerabilities fixed ↑	62.2%	63.5%	83.7%	**97.1%**
Build ↑	87.0%	**98.4%**	98.0%	96.7%
Tests pass ↑	80.5%	98.7%	**99.0%**	95.4%
All gates pass (PASS) ↑	46.9% (144/307)	59.6% (183/307)	77.5% (238/307)	**90.2%** (277/307)
Vulnerability instances fixed ↑	56.5% (1,168/2,066)	72.7% (1,503/2,066)	83.1% (1,716/2,066)	**95.9%** (1,982/2,066)

The individual gates show a trade-off. Evo Remediation Agent never leaves dependencies unresolved and fixes every targeted vulnerability in 97.1% of upgrades, but its build and test rates trail Claude Code + Snyk scan's by 1.3 and 3.6 percentage points. On dependency resolution, build and tests combined, the agents perform similarly (92.8% for Evo Remediation Agent and Claude Code + Snyk scan, 93.8% for Claude Code unaided). Claude Code's high build and test rates partly reflect avoidance: it often leaves the vulnerability in place, and an untouched dependency rarely breaks anything (Section 5.3). The package manager bump falls short on both halves: it fixes the targeted vulnerabilities about as often as Claude Code unaided (62.2%), but keeps the project working in only 64.2% of upgrades, because it never repairs what the upgrade breaks.

The ranking holds across repositories and package managers. Macro PASS rates, which weight every repository equally, are 51.3%, 68.0%, 80.1% and 93.1% for the package manager bump, Claude Code unaided, Claude Code + Snyk scan and Evo Remediation Agent, and every pairwise difference between the agents is statistically significant (Section 4.5). Evo Remediation Agent's macro PASS rate is at or above both Claude Code agents in every package manager, although several have only three repositories.

5.2 Snyk's Guidance Secures the Easy Half; Agents Separate on the Rest

Table 3: Agentic harnesses on 144 Snyk-advice-solvable upgrades, which the package manager bump passes, and on the other 163, which it fails. The split is a lower bound on what mechanical upgrading can solve (Section 5.3). *Regressions*: the package manager bump passes, the agent fails. *Gains*: the agent passes, the package manager bump fails.

	Claude Code unaided	Claude Code + Snyk scan	Evo Remediation Agent
PASS on Snyk-advice-solvable (144) ↑	86.1%	95.8%	**99.3%**
Regressions ↓	20	6	**1**
PASS on other upgrades (163) ↑	36.2%	61.3%	**82.2%**
Gains ↑	59	100	**134**

Neither Snyk's guidance nor an agent is enough on its own. Applied by the package manager bump, without an agent, Snyk's recommended upgrade passes 46.9% of upgrades; Claude Code

without Snyk passes 59.6%; given the Snyk scan and recommended upgrade, Claude Code passes 77.5% (Table 2). The split shows what each contributes (Table 3); per-repository (macro) rates give the same ranking on both splits.

On the 144 Snyk-advice-solvable upgrades, what matters is knowing which version is safe. With Snyk's guidance, the agents rarely regress (Claude Code + Snyk scan on 6 upgrades, Evo Remediation Agent on 1); without it, Claude Code unaided regresses on 20, in 15 of them by upgrading to a version Snyk still flags. Finding a fixed version by web search, e.g. on OSV, proved unreliable; an SCA scan names the fixed version and can confirm the upgrade. The pattern holds across all upgrades: Claude Code unaided chose a still-vulnerable version 64 times, Claude Code + Snyk scan 13 times (Table 4). Vulnerability databases can disagree on which version fixes a vulnerability, and the rescan uses Snyk Open Source, so part of the gap may reflect such disagreement (Section 7).

On the other 163 upgrades, which need more than a version bump, the agent decides the outcome: PASS rates are 82.2%, 61.3% and 36.2% for Evo Remediation Agent, Claude Code + Snyk scan and Claude Code unaided, and these upgrades supply 34 of Evo Remediation Agent's 39 extra PASSes over Claude Code + Snyk scan.

5.3 The Agents Fail in Opposite Ways

Claude Code mostly stops short of the fix; Evo Remediation Agent lands the upgrade and sometimes breaks the code (Figure 4). The rescan is the first failed gate in 107 of Claude Code unaided's 124 failures and 48 of Claude Code + Snyk scan's 69, but in only 9 of Evo Remediation Agent's 30; 21 of Evo Remediation Agent's failures break the build or tests, and on 11 of these upgrades Claude Code + Snyk scan left the vulnerability in place. Table 4 gives the root causes.

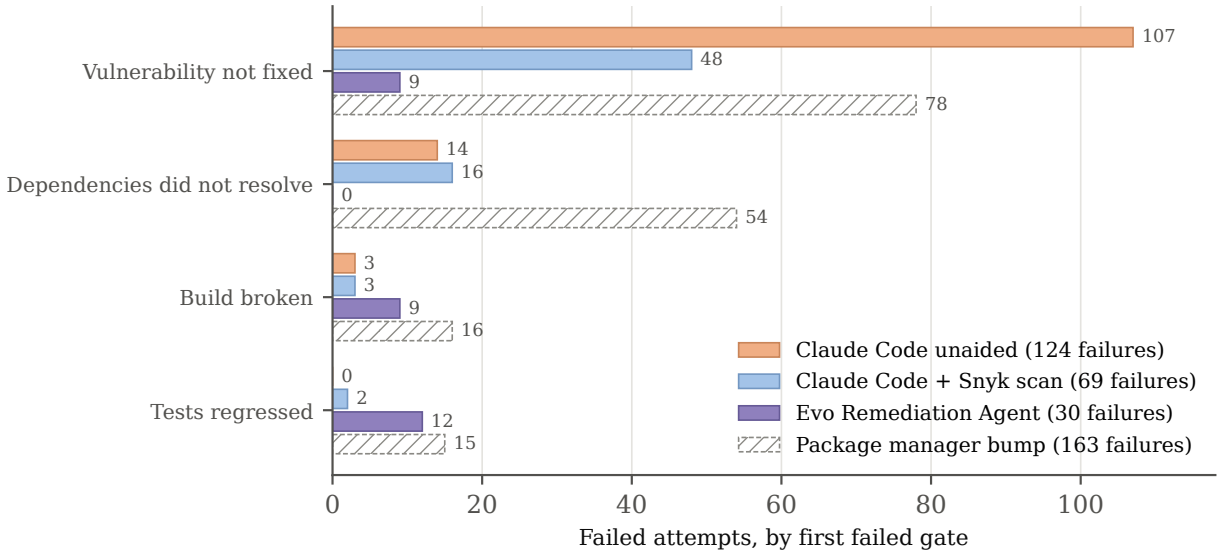


Figure 4: First failed verification gate for every failed attempt (Package manager bump: 163 failures, Claude Code unaided: 124, Claude Code + Snyk scan: 69, Evo Remediation Agent: 30).

Claude Code fails silently. It often avoids the risky upgrade: instead of the major version that fixes every targeted vulnerability, it settles for a patch or minor upgrade that fixes only some vulnerabilities. Claude Code unaided stayed within the current major version in 30 attempts, and even when told the upgrade target version, Claude Code + Snyk scan did so in 10 (e.g. `spring-`

`boot-starter-test` 2.7.18 instead of 3.5.15). Claude Code + Snyk scan also pinned or excluded transitive packages instead (15) or made no change (12). In 70 of its 124 failures, Claude Code unaided claimed to have fixed vulnerabilities that the rescan still found. Other fixes are incomplete outside the agent's session:

- **Unsaved dependency changes.** Some fixes worked in Claude Code's session but not in a clean install: in `bulletproof-nodejs`, Claude Code + Snyk scan left `package.json` and `package-lock.json` out of sync, and in `express-typescript-starter`, Claude Code installed with `--legacy-peer-deps` without saving the flag. Of the 30 Claude Code attempts that failed the strict dependency gate, 19 passed every other gate; Evo Remediation Agent never failed it.
- **Interpreter-specific fixes.** In 15 Claude Code unaided and 8 Claude Code + Snyk scan failures, the fixed release requires a newer Python than the project targets (e.g. `filelock` 3.20.3 requires Python 3.10); the agent locked the fix only for newer interpreters, leaving the project's own on the vulnerable version.

Table 4: Root cause of every failed attempt, grouped by cause rather than by first failed gate, so counts can differ from Figure 4. *Vulnerability remediation*: the targeted vulnerability remains. *Version compatibility*: the new versions violate a version the project pins or requires. *Code compatibility*: changed API or behaviour breaks the project. *Other*: a package removed from PyPI, a source build failure and a failed rescan.

Cause	Package manager bump	Claude Code unaided	Claude Code + Snyk scan	Evo Remediation Agent
Vulnerability remediation	77	92	40	8
Upgraded to still-vulnerable version	2	64	13	1
Still reachable via another path	75	13	15	3
No upgrade made, or reverted	0	15	12	4
Version compatibility	54	30	25	7
Resolver rejects dependency versions	25	14	16	0
Fix requires newer Python	26	15	8	1
Dependencies need a newer compiler	3	1	1	6
Code compatibility	28	2	4	15
Changed API breaks build	13	2	2	3
Class or method missing at runtime	12	0	1	6
Changed behaviour fails tests	3	0	1	6
Other	4	0	0	0
Failed attempts	163	124	69	30

Evo Remediation Agent fails visibly. It attempts the upgrades Claude Code often avoids, so half its failures (15 of 30) are code incompatibilities, 13 of them on major upgrades, such as Spring 6's move from `javax.*` to `jakarta.*`. It also sometimes upgrades more than needed: in two TypeScript repositories, a broad lockfile regeneration moved transitive type definitions beyond what the project's older compiler can parse (6 failures), and its only regression came from a caret range (`agenda@^4.0.0`) that floated to a new major version of the MongoDB driver.

The package manager bump fails at every gate. It always targets the recommended version but never repairs what the upgrade breaks: 54 of its 163 failures first fail at dependency resolution and 31 at the build or tests. In 75, the vulnerability stays reachable through another path, mostly in managed builds, where pinning only the named package leaves the rest of its BOM-managed family at old versions. The Snyk-advice-solvable split is therefore a lower bound on what mechanical upgrading can solve.

5.4 Major Framework Migrations Remain Hard

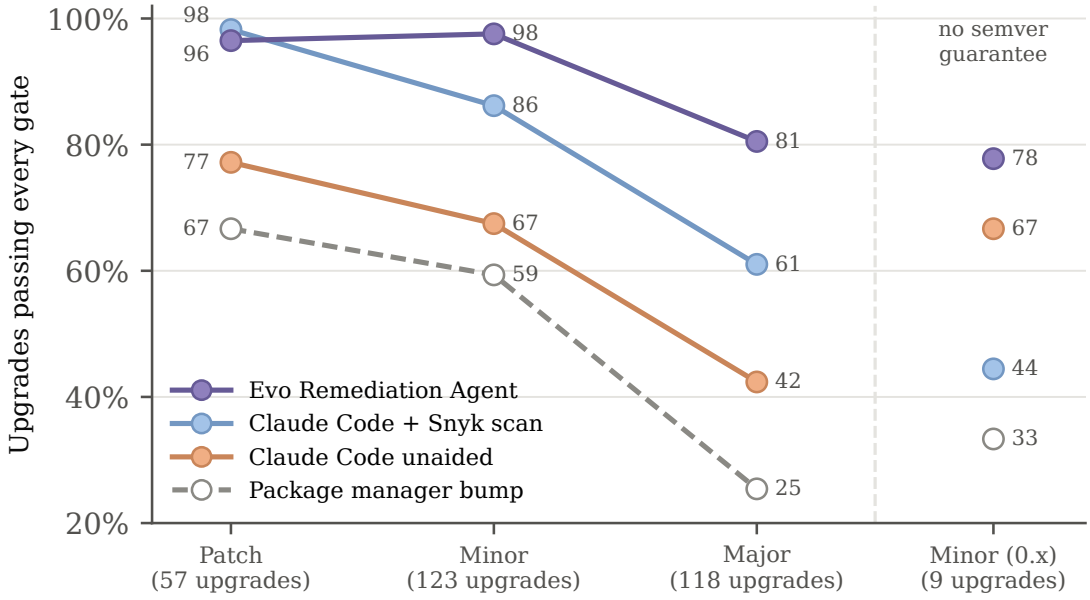


Figure 5: Share of upgrades passing every gate, by the size of the recommended version jump (not the smallest fixing jump of Section 3.1). Minor jumps within 0.x versions are shown separately because semantic versioning gives no compatibility guarantee below 1.0.

Major upgrades separate the agents most (Figure 5): PASS rates fall from 96%, 88% and 70% on the 189 non-major upgrades to 81%, 61% and 42% on the 118 major ones for Evo Remediation Agent, Claude Code + Snyk scan and Claude Code unaided. The package manager bump passes 67%, 58% and 25% of patch, minor and major upgrades.

Major upgrades are where code changes are needed. Evo Remediation Agent edited non-test application source in 52% of its 95 major PASSes, against 5% of its 182 non-major ones; Claude Code did so less often (Claude Code + Snyk scan 32% and 3%, Claude Code unaided 24% and 2%). They are also where headroom remains: 23 upgrades in seven Maven and npm repositories defeated all three agents, mostly major framework migrations (Spring 5→6/7, Spring Boot 2→3/4, Dropwizard 2→5).

Smaller jumps are not always safe: 10 of the 189 patch and minor upgrades broke the build or tests for at least one agent, e.g. `logback-classic` 1.2.11→1.6.3 (a binary incompatibility) and `flask` 3.0.3→3.1.3. This is consistent with semantic-versioning studies (Section 2) and is why VulnBump verifies every fix against the build and tests, whatever the jump.

5.5 Transitive Vulnerabilities Are Harder for Every Harness

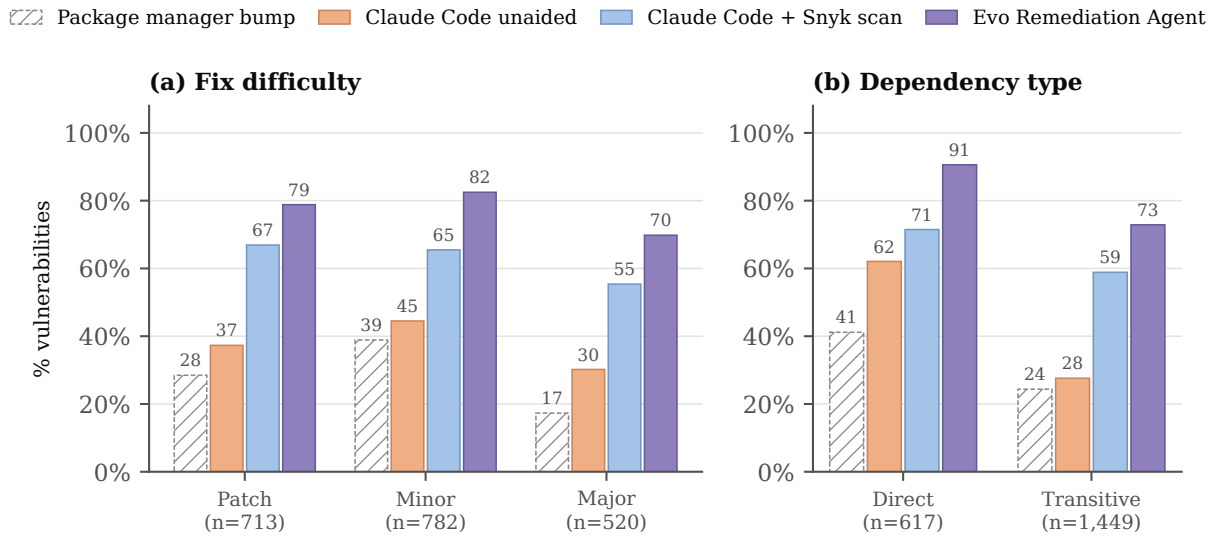


Figure 6: Share of fixable vulnerabilities fixed by passing upgrades, by (a) fix difficulty (smallest semantic-versioning jump to a fixed release) and (b) direct or transitive dependency; 51 vulnerabilities without a fix difficulty are omitted from (a).

Transitive vulnerabilities make up 70% of fixable instances, yet for every harness, passing upgrades fix a smaller share of them than of direct ones (Figure 6). Snyk's guidance matters most here: it more than doubles the share of transitive instances Claude Code fixes, from 28% to 59%, against 73% for Evo Remediation Agent; on direct instances, Claude Code fixes 62% without it.

5.6 New Vulnerabilities

Upgrades can pull in packages with their own known vulnerabilities, which PASS does not penalise (Section 7). Post-fix scans show new instances in 62 Evo Remediation Agent attempts (across 16 repositories), 30 Claude Code + Snyk scan and 33 Claude Code unaided attempts.

6 Discussion

Dependency upgrades and vulnerability fixes must be evaluated jointly. Scored on dependency resolution, build and tests alone, the agents perform similarly, because leaving a vulnerability in place rarely breaks anything; scored on the vulnerability alone, Evo Remediation Agent's breakages would go unnoticed. A breaking-update benchmark, which checks no vulnerability, therefore cannot observe avoidance, Claude Code's most common failure.

An SCA scanner is a key tool for agents fixing vulnerabilities. Breaking-update benchmarks already show that agents can partly repair what an upgrade breaks, by adapting code and migrating APIs (Section 2); our results show that agents also need vulnerability data. Given the Snyk scan and recommended upgrade, Claude Code more than halves the attempts whose first failed gate is the rescan (107 to 48). Without them, it regressed on 20 upgrades that Snyk's recommended upgrade alone solves, mostly by choosing a version that is still vulnerable, and its passing upgrades

fixed less than half as large a share of transitive as of direct vulnerabilities. Web search does not reliably tell an agent which version is safe, least of all for packages the project does not declare directly.

Knowing the target is not enough. With the same findings and target, Evo Remediation Agent passes 39 more upgrades than Claude Code + Snyk scan, 23 of them major, and rarely shows Claude Code + Snyk scan's main failures: stopping within the current major version and pinning transitive packages instead (Section 5.3). Evo Remediation Agent combines Snyk intelligence with a loop in which it builds, tests and repairs (Section 4.2), and our setup cannot isolate which component makes the difference. That the system around the model can matter this much is consistent with DepRepair, where the full pipeline (distilled migration rules plus localisation and per-category guidance) raised pass rates by 7–11 points while raw upstream evidence lowered them by 7–23 points [23], and with DepBench, where changing only the harness moved results by up to 16.3 points [10].

Failure costs and remaining headroom. Claude Code's failures are mostly silent: the vulnerability was left unfixed, and in 70 of its 124 failures, Claude Code unaided reported it fixed. Most of Evo Remediation Agent's failures are visible in the project's own build and tests, where CI or a reviewer will see them. All agents still struggle with major framework migrations: 23 upgrades, mostly Spring, Spring Boot and Dropwizard migrations, defeated all three, and half of Evo Remediation Agent's major PASSes needed application-source edits. VulnBump therefore leaves room for better agents, and major migrations may still warrant human review.

7 Threats to Validity

Construct validity. A PASS is only as reliable as the SCA scanner and the project's tests.

- **Snyk Open Source as task source and judge.** Tasks and the rescan use Snyk's vulnerability database, which Claude Code unaided cannot consult, so a fixed version taken from another database may still be flagged. This penalises only Claude Code unaided; Evo Remediation Agent and Claude Code + Snyk scan receive the same findings and target.
- **Test strength.** Breaking changes, especially changes in behaviour, are caught only as well as the project's tests exercise them. Six repositories (91 of the 307 upgrades) have at most four passing baseline tests, leaving most of the work to the build gate. VulnBump also relies only on each project's own tests, which agents can run, with no held-out tests or tests generated to target the upgrade. Held-out tests catch failures that visible ones miss: in DepBench, 61.8% of failed attempts had passed the visible tests or build [10]. Even purpose-built tests catch only part: BreakGuard, which generates tests specifically to detect breaking dependency updates, catches at best 30.3% of those in the BUMP benchmark [15].
- **Verifier noise.** Test identities omit the module name, so same-named tests in different modules can mask regressions; three npm repositories without lockfiles let dependency trees float between baseline and verification; and the baseline is measured once, so a flaky test can be recorded as a regression (BUMP reruns tests three times [17]).
- **A deliberately simple baseline.** The package manager bump never moves a BOM or parent or tries another version, so the Snyk-advice-solvable split is a lower bound on what

mechanical upgrading can solve, and the agents' gains overstate what only an agent can do (Section 5.3).

- **New vulnerabilities not penalised.** They appear in more Evo Remediation Agent attempts than Claude Code attempts (Section 5.6), so this favours Evo Remediation Agent.

Internal validity.

- **Leakage.** The GitHub and Snyk blocks are host-file based, without egress filtering, so DNS-over-HTTPS, hard-coded IPs, unlisted mirrors, web archives and search snippets can bypass them; more simply, about 25 repositories are published libraries whose later releases are on package registries. Leakage would favour Claude Code, since Evo Remediation Agent has no web tool and its prompt forbids such fetches; Claude Code transcripts were not kept, so leakage cannot be audited.
- **Training-data exposure.** 40 of the 43 public repositories are pinned at commits that predate Claude Sonnet 5.5's training data cutoff (June 2026) [3], so the model may have seen these projects, their later upgrades or fixes for the same vulnerabilities. This may inflate absolute pass rates, but all three agents share the model.

External validity.

- **Case set selection.** A few very large repositories, such as Apache Kafka and Spring Framework, were excluded because they exhausted memory or time in earlier runs or had hundreds of upgrades that would take days to evaluate one by one, so VulnBump under-represents very large repositories and, since several are Gradle builds, Gradle.
- **Scope.** VulnBump covers the six package managers behind about 92% of production issue–project pairs (not Go, .NET, Ruby, PHP or pnpm), and scores upgrades one at a time rather than ordered and combined in one tree, as remediation is usually run in practice.

Conclusion validity. Each agent, and the package manager bump, ran once per upgrade, so the significance tests capture variation across repositories, not reruns. Upgrades are concentrated: the three largest repositories contribute 27% of them, and single repositories dominate some failure categories (e.g. one accounts for 28 of the 30 Claude Code dependency-gate failures), so the pooled counts of Section 5.3 lean on a few projects, whereas macro rates weight repositories equally. All counts are a snapshot of the vulnerability database at scan time (Section 8).

Future work. The planned re-run of VulnBump adds repeated runs to measure run-to-run variance; three baseline test runs to detect flakiness; held-out tests that agents cannot run, to catch changes in behaviour; a whole-repository mode that applies all of a repository's upgrades together instead of each in isolation; a verdict that also penalises new vulnerabilities; egress-level leakage controls; further ecosystems such as Go, .NET and pnpm; more models, to separate the effect of the model from that of the agent; and retained transcripts, for failure-cause adjudication by several annotators with measured agreement.

8 Data Availability

The VulnBump harness (task builder, sandbox, verifier and analysis scripts) and the 43 evaluation cases (repository, pinned commit, image and commands) are open source at https://github.com/snyk-labs/snyk-vulnbump. Tasks are derived from a Snyk scan at run time, which requires the Snyk CLI and a token. The repositories are pinned but the vulnerability database is not, so a later run on the same commits will usually find more than our 2,066 fixable instances (scanned 4–5 October 2026) and may recommend different targets. For example, CVE-2026-76183 (high severity), published between our last agent scan (5 October) and the package manager bump run (8 October), is fixed only from `tomcat-embed-websocket` 10.1.60, so two upgrades in tokuhirom/java-handbook target 10.1.60 for the package manager bump but 10.1.59 for the agents. Runs scanned on different days should therefore be compared only on tasks with the same recommended upgrade, and against their own baseline.

9 Conclusion

VulnBump evaluates vulnerability-driven dependency upgrades in 43 real repositories across six package managers, matched to Snyk Open Source production data, with a joint verdict checked independently in a fresh container: the vulnerability must be fixed and the project must still resolve, build and pass its tests. On 307 upgrades with the same model, this verdict separates agents that a breakage-only check ranks equal: Evo Remediation Agent passes 90.2% of upgrades, against 77.5% for Claude Code + Snyk scan and 59.6% for Claude Code unaided. Claude Code more often leaves the vulnerability in place while Evo Remediation Agent more often lands the upgrade, occasionally breaking the project. An SCA scanner is a key tool for remediating vulnerable dependencies: Snyk's guidance raises Claude Code's PASS rate by 17.9 percentage points, and without it Claude Code often settles on a version that is still vulnerable. Yet knowing the target is not enough: given the same Snyk findings and model, Evo Remediation Agent's purpose-built harness passes 12.7 points more than Claude Code + Snyk scan. Major framework migrations remain hard for every agent. Repeated runs, held-out tests and a whole-repository mode are the next steps towards a more reliable and realistic evaluation of vulnerability-driven dependency upgrades.

References

[1] Anthropic. Claude Code documentation. https://code.claude.com/docs/en/overview. Accessed: 2026-10-06.

[2] Anthropic. Claude Code tools reference. https://code.claude.com/docs/en/tools-reference. Accessed: 2026-10-08.

[3] Anthropic. Models overview. https://platform.claude.com/docs/en/models/overview. Accessed: 2026-10-07.

[4] Daniele Cipollone, Sergey Titov, Maliheh Izadi, Egor Bogomolov, and Arie van Deursen. LibEvoBench: Probing temporal knowledge stratification in code generation models, 2026. arXiv:2606.25402. DL4Code Workshop at ICML 2026.

[5] Anthony C. Davison and David V. Hinkley. *Bootstrap Methods and their Application*. Cambridge University Press, 1997.

[6] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Huashan Lei, Guanping Xiao, Yepang Liu, and Zheng Zheng. PCREQ: Automated inference of compatible requirements for Python third-party library upgrades. *ACM Transactions on Software Engineering and Methodology*, 2026.

[9] Wenke Li, Feng Wu, Cai Fu, and Fan Zhou. A large-scale empirical study on semantic versioning in Golang ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1604–1614, 2023.

[10] Zijian Luo, Runzhi He, Pengfei Gao, Yu Kang, Zeqi Lin, Minghua Ma, Qingwei Lin, Saravan Rajmohan, and Yongqiang Tian. Update from hell: Can coding agents survive hidden breakage in dependency upgrades? 2026. arXiv:2608.30300.

[11] Diganta Misra, Nizar Islah, Victor May, Brice Rauby, Zihan Wang, Justine Gehring, Antonio Orvieto, Muawiz Sajjad Chaudhary, Eilif B. Muller, Irina Rish, Samira Ebrahimi Kahou, and Massimo Caccia. GitChameleon 2.0: Evaluating AI code generation against Python library version incompatibilities. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 46792–46831, 2026.

[12] Lina Ochoa, Thomas Degueule, Jean-Rémy Falleri, and Jurgen Vinju. Breaking bad? Semantic versioning and impact of breaking changes in Maven Central: An external and differentiated replication study. *Empirical Software Engineering*, 27(3):61, 2022.

[13] Open Source Vulnerabilities. OSV: A distributed vulnerability database for open source. https://osv.dev/. Accessed: 2026-10-06.

[14] Steven Raemaekers, Arie van Deursen, and Joost Visser. Semantic versioning and impact of breaking changes in the Maven repository. *Journal of Systems and Software*, 129:140–158, 2017.

[15] Rachna Raj, Benoit Baudry, and Diego Elias Costa. BreakGuard: Towards detecting dependency breaking changes with LLM-generated tests, 2026. arXiv:2608.20167.

[16] Frank Reyes, Benoit Baudry, and Martin Monperrus. Agentic generation of AST transformation rules for fixing breaking updates, 2026. arXiv:2606.24446.

[17] Frank Reyes, Yogya Gamage, Gabriel Skoglund, Benoit Baudry, and Martin Monperrus. BUMP: A benchmark of reproducible breaking dependency updates. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 159–170, 2024.

[18] Frank Reyes, May Mahmoud, Federico Bono, Sarah Nadi, Benoit Baudry, and Martin Monperrus. Byam: Fixing breaking dependency updates with large language models. *Empirical Software Engineering*, 31(4):113, 2026.

[19] Snyk. Fix your vulnerabilities. https://docs.snyk.io/scan-fix-and-prevent/scan-with-snyk/snyk-open-source/manage-vulnerabilities/fix-your-vulnerabilities. Accessed: 2026-10-09.

[20] Snyk. Snyk CLI documentation. https://docs.snyk.io/developer-tools/snyk-cli/overview. Accessed: 2026-10-06.

[21] Snyk. Snyk Open Source documentation. https://docs.snyk.io/scan-fix-and-prevent/scan-with-snyk/snyk-open-source. Accessed: 2026-10-08.

[22] Vali Tawosi, Salwa Alamir, Xiaomo Liu, and Manuela Veloso. LLM agents for automated dependency upgrades. In *AISM Workshop at the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.

[23] Shenghao Yang, Bo Lu, Yaochen Liu, Yu Kang, Qiongfang Zhang, Chetan Bansal, Saravan Rajmohan, and Minghua Ma. DepRepair: LLM-based source-code repair for dependency breaking changes, 2026. arXiv:2607.17957.

[24] Lyuye Zhang, Chengwei Liu, Zhengzi Xu, Sen Chen, Lingling Fan, Lida Zhao, Jiahui Wu, and Yang Liu. Compatible remediation on vulnerabilities from third-party libraries for Java projects. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 2540–2552, 2023.